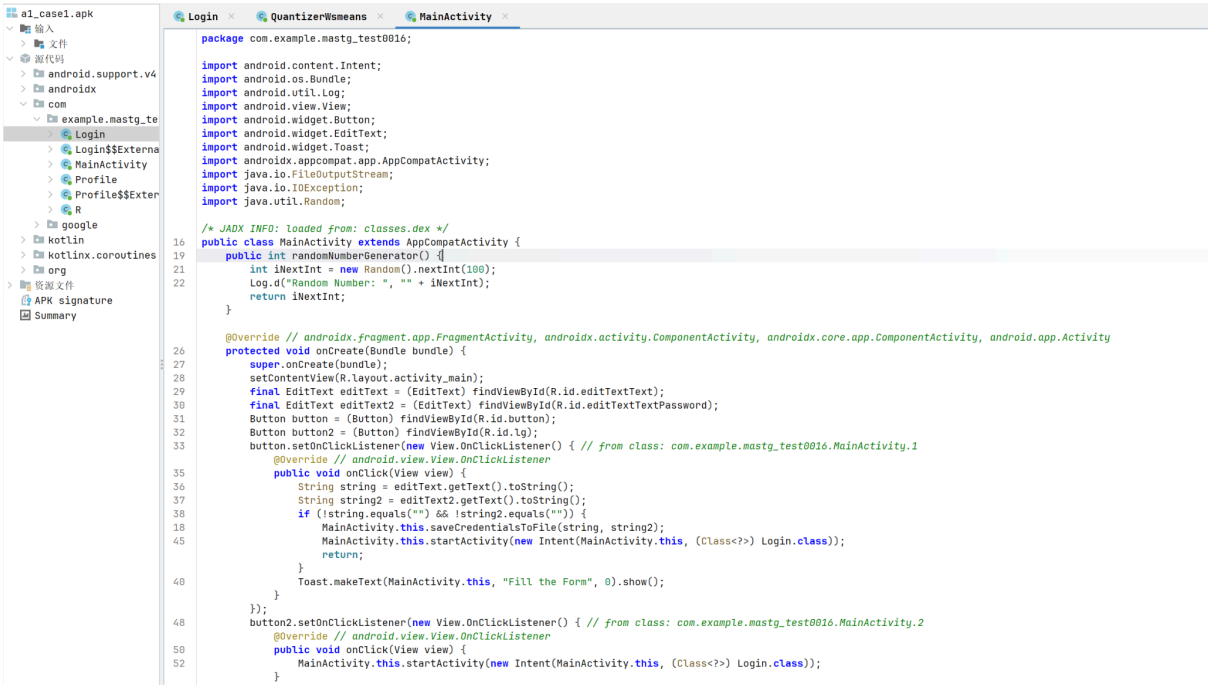
AI-Assisted Cryptographic Vulnerability Discovery in Android APK

1. Introduction

Android applications are typically distributed as APKs (Android application installers), which contain compiled code and resource files; the original source code is not directly visible. Therefore, during security audits, decompilation tools are needed to restore this content to a more readable form. In this project, I used Jadx to decompile the provided APK file to examine its internal code logic. The purpose of this was to identify vulnerabilities in security-sensitive aspects of the application, such as random number generation, and to assess the potential security risks posed by these vulnerabilities.



---

2. System and Threat Model

System Overview

The application allows users to input a username and password and stores these credentials locally. After registration, users can navigate to a login interface. The system consists of:

- User interface (input fields and buttons)
- Application logic (credential handling)

- Local storage (file system)

Assets

The primary sensitive assets include:

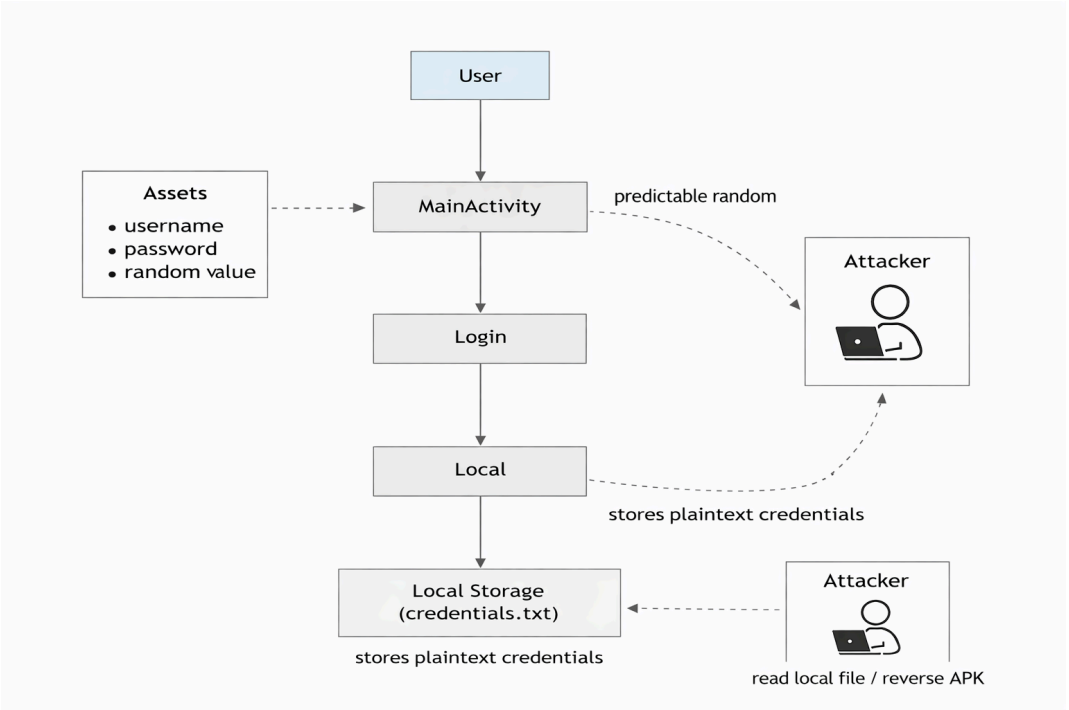
- User credentials (username and password)
- Any values generated for identification or authentication

Threat Model

We assume an attacker with the following capabilities:

- Reverse engineering the APK using Jadx
- Access to local storage (e.g., via ADB or rooted device)
- Ability to observe or approximate execution timing

The attacker's goal is to predict security-sensitive values or extract sensitive information for unauthorized access.



---

3. Methodology

The APK was decompiled using Jadx, and static analysis was performed on the reconstructed Java source code. We searched for security-sensitive operations using keywords such as:

- “Random”
- “token”
- “password”

AI tools were used to assist in identifying insecure coding patterns and validating whether specific APIs (e.g., `java.util.Random`) are appropriate for security-sensitive contexts.

---

4. Vulnerability Analysis

4.1 Insecure Random Number Generation (Primary Vulnerability)

The application generates random values using:

```
int iNextInt = new Random().nextInt(100);
```

Issue

The use of `java.util.Random` is not suitable for security-sensitive operations. This class produces pseudo-random numbers based on a predictable seed, typically derived from the system time.

Security Risk

If such values are used in authentication, session identifiers, or verification codes, an attacker can predict future outputs. This violates the requirement for unpredictability in security-sensitive randomness.

---

5. Exploitation Scenario

The vulnerability can be exploited as follows:

1. The attacker reverse engineers the APK using Jadx and identifies the use of `java.util.Random`.
2. The attacker observes or approximates the time at which the random value is generated.
3. Since `Random` is seeded using system time, the attacker brute-forces possible seed values within a small time window.
4. The attacker reproduces the same sequence of random numbers locally.
5. If the random value is used in a security context (e.g., token or verification code), the attacker can predict it and bypass authentication.

This demonstrates that the randomness used is not secure against adversarial prediction.

---

6. Additional Observation: Insecure Credential Storage

The application stores credentials in plaintext:

("Username: " + username + " Password: " + password)

This exposes sensitive data to attackers with local storage access. While this issue is not directly related to randomness or cryptographic misuse, it significantly increases the overall security risk.

---

7. Mitigation

Fix for Randomness

Replace `java.util.Random` with a cryptographically secure alternative:

SecureRandom secureRandom = new SecureRandom();

`SecureRandom` generates unpredictable values suitable for security-sensitive operations.

Fix for Credential Handling

- Avoid storing credentials in plaintext
- Use secure hashing (e.g., SHA-256 or bcrypt)
- Use Android secure storage mechanisms (e.g., Keystore)

---

8. Conclusion

This analysis identified a critical misuse of randomness in the application, where `java.util.Random` is used in a context that may require secure unpredictability. This allows attackers to predict generated values and potentially bypass security mechanisms. By replacing insecure randomness with cryptographically secure alternatives and improving data handling practices, the application's security posture can be significantly improved.